

EvoArena / EvoMem — 动态环境下的 Agent 记忆进化

全称: EvoArena: Tracking Memory Evolution for Robust LLM Agents in Dynamic Environments
arXiv: 2606.13681 (2026.06, v2) | 机构: **新加坡国立大学 NUS
关键词: Memory Evolution · Dynamic Environments · Patch-based Memory · Progressive Updates · Step / Chain-level Accuracy · Version-aware Reliability

1. 这篇论文为什么重要

一句话: EvoArena 是首个把"环境随时间演化"作为核心评测维度的基准——它把环境变化建模为一系列**渐进式更新**, 并提出 **EvoMem** (基于 patch 的记忆范式) 让 agent 通过"记忆的变化"来推理"环境的演化".

它戳破了一个评测盲区: **绝大多数 agent 评测假设环境是静态的**。但真实部署本质是**动态的**——agent 必须随环境和任务条件的变化, 持续对齐自己的知识、技能与行为。

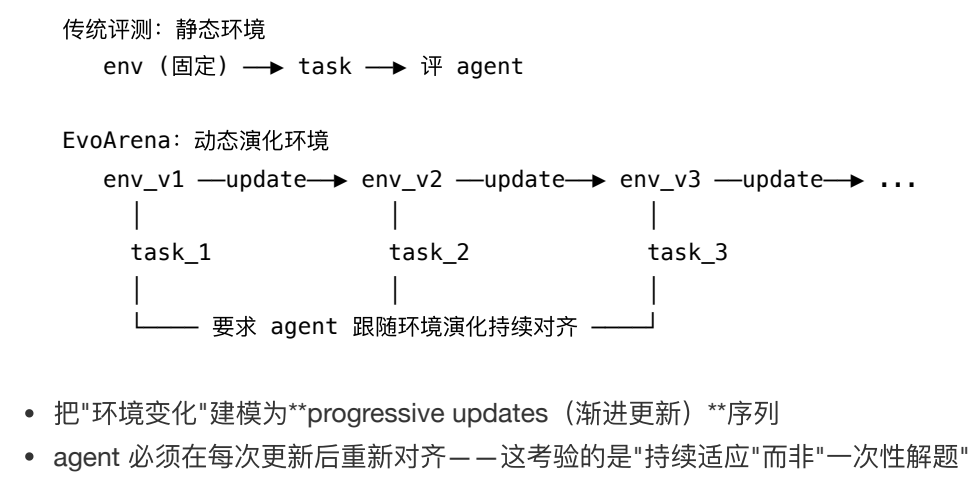
这篇论文从两个方面填补空白:

- **评测侧**: EvoArena 基准——把环境变化建模为 terminal / software / social 三域上的**渐进更新序列**, 且每个域都是在现有基准上“演化化”改造 (Terminal-Bench-Evo / SWE-Chain-Evo / PersonaMem-Evo)
- **方法侧**: EvoMem——把**记忆进化**记录为结构化的“追加式补丁历史 (append-only patch history)”, 让 agent 能推理环境演化

这是把"evolve"的视角从"agent 能力进化"转向**"环境演化 + 记忆进化"**的代表作, 也是本库时间跨度最新的一篇 (6 月)。

2. 核心方法

2.1 EvoArena 基准: 环境作为"渐进更新序列"

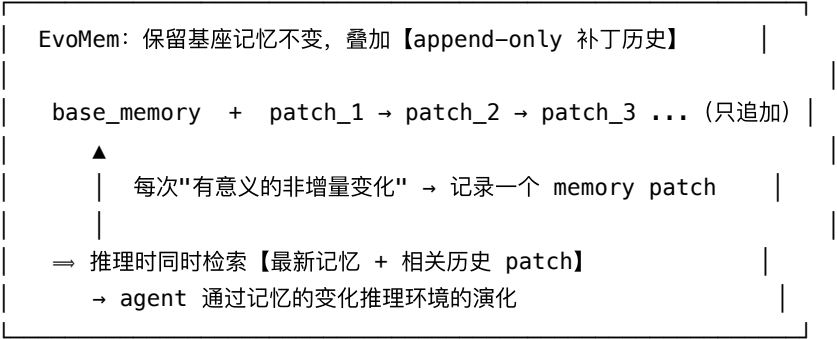


- 共同挑战是 **version-aware reliability**（版本感知可靠性）：既要适应新条件，又要保留仍然有效的旧行为

三个演化领域（各自把现有基准"演化化"改造）：

子基准	演化类型	评测的 Agent	演化内容
Terminal-Bench-Evo	可执行工作流演化	Terminus 2	终端任务变成离散版本链：终点不变，但后续版本改变部署机制、路径、权限、分支策略、依赖、测试
SWE-Chain-Evo	软件演化	OpenHands	仓库里程碑按时间顺序评测： 每个新需求在累积的代码状态上解决，用 Pass-to-Pass 测试 检查旧行为是否仍成立
PersonaMem-Evo	社交 / 偏好演化	A-Mem	长对话史里含 隐式且演化的偏好证据 ：agent 要按当前偏好状态作答，并区分"已过时但有历史依据"的旧证据

2.2 EvoMem：基于 Patch 的记忆进化范式



三步机制：

- Record Changes**（记录变化）——只捕捉**非增量更新（non-additive）**，每个 patch 记录五要素：
previous state (旧状态) / new state (新状态) / rationale (原因) / summary (摘要) / evidence (证据)
- Retrieve Versioned Evidence**（检索带版本的证据）——当查询依赖演化时，把相关历史 patch 与当前记忆一并返回
- Preserve Agent Interfaces**（保留 agent 接口）——同一抽象可无侵入地实例化到 **Terminus 2 / OpenHands / A-Mem / Memento-Skill** 四种现有 agent 上

核心思想：

- 不是"覆盖"旧记忆，而是把记忆维护成 **base memory + 只追加的 patch 序列**——一份结构化更新历史
- agent 看的是"记忆怎么变的"，从而推断"环境怎么变的"
- 这保留了**完整的演化状态**，而非只有最新快照——避免"单一合并记忆状态"丢失历史证据

2.3 深入：base_memory 与 patch

EvoMem 的一句话定位是 "git-like memory" (类 git 的记忆)：base_memory 是"工作区当前快照", patch 历史是"commit 日志"。

① base_memory (最新合并记忆 M_t)

- 存什么：base agent 自己维护的**最新合并状态**——可以是文本记忆库、用户画像、记忆图、或一份 skill 文件。EvoMem 完全不改它。
- 怎么更新：由 base agent 原本的更新函数 U 负责： $M_t = U(M_{t-1}, x_t)$ (x_t 是 t 时刻的观测)。
- 怎么用：推理时默认只用它—— $c_{\text{mem}} = R_{\text{mem}}(q, M_T)$ ，即用原检索器从最新记忆取证据。够用就到此为止。
- 局限：只保留"最新信念", 旧状态被覆盖即丢——这正是 patch 要补的洞。

② patch (补丁 p_t) —— 重点

- 何时创建：EvoMem 监控 $M_{t-1} \rightarrow M_t$ 的转移，算出差异 $\Delta_t = \text{Diff}(M_{t-1}, M_t)$ 。**只对非增量更新** (revise / overwrite / reinterpret 既有记忆) 创建 patch；**纯新增** (additive) 的信息留在 base_memory 里，不建 patch。
- 存什么 (6 个字段)： $p_t = (\tau_t, C_t^-, C_t^+, r_t, z_t, e_t)$

字段	含义	说明
τ_t	时间元数据	turn / session / timestamp, 用于按时间排序
C_t^-	更新前的 受影响记忆内容	旧状态 (pre-update)
C_t^+	更新后的 受影响记忆内容	新状态 (post-update)
r_t	更新原因 (rationale)	为什么改
z_t	变化的语义摘要	一句话概括
e_t	支撑证据 (evidence)	触发的交互 / 任务上下文 / 执行反馈 / 环境快照

- 怎么存：写入只追加 (**append-only**) 的 **patch 历史** $\mathcal{P}_{1:t} = \{p_1, \dots, p_t\}$ ，与 base_memory **分开存**。于是 M_t 保存"当前合并态", $\mathcal{P}_{1:t}$ 保存"它是怎么一步步变成现在这样的"。
- 怎么用 (**检索 + 拼接**)：给定查询 q ,
 - i. 先按常规取最新记忆证据 $c_{\text{mem}} = R_{\text{mem}}(q, M_T)$;
 - ii. 当 query 依赖"被覆盖的旧状态 / 时序变化 / 版本相关行为"时，再取 top- k 相关 patch: $\mathcal{P}_q = R_{\text{patch}}(q, \mathcal{P}_{1:T})$;
 - iii. 拼成最终上下文 $c(q) = \text{Concat}(c_{\text{mem}}, \mathcal{P}_q)$ 喂给 agent。检索到的 patch **按时间排序** (越晚 = 越新的证据)。

③ 怎么判断"非增量更新" (建不建 patch 的判据)

核心：先对记忆做 diff，再看这次改动是"动了已有内容"还是"纯加新内容"——只有前者建 patch。

- 通用流程 (§4.2)：EvoMem 旁路监控 base 更新函数 U 产生的转移，算 $\Delta_t = \text{Diff}(M_{t-1}, M_t)$ 。
 - 非增量 (**建 patch**)：revise / overwrite / reinterpret 既有记忆——改动触及了已存在的条目/字段/关系。
 - 纯增量 (**不建**)：只追加全新条目、没动旧内容——这类信息本就保留在 base_memory 里。
 - 一句话判据：改动是否"踩到"了已有记忆。
- 各 agent 的具体判据 (附录 F)：

Agent	diff 对象	判为"非增量"的条件
A-Mem (F.4)	新建/被改 note、 变化字段、改动 link	overwrite_update (note 内容/ 元数据被修订)、link_rewrite_update (已有 note 邻接被重连)； 纯加新 note 不建；并抑制 assistant 轮，只留用户驱动的演化
Terminus 2 (F.1)	相邻任务指令差 $\delta_t = \text{Diff}(x_{t-1}, x_t)$	从指令差异+环境信号+执行证据推断变更类型 c_t (输入源/输出路径/ 格式契约/工作目录/工具链改变)；链上 首个任务只写 ledger、不建 patch
OpenHands (F.2)	改动文件/代码 hunk 分成的特征级单元	后续任务 推翻早先实现假设 (覆盖旧逻辑) 时建记录
Memento-Skill (F.3)	tip 版本比对	仅当出现 可复用学习信号 (答错→judge 反馈→轨迹分析) 才写新版本 tip

判据来自 **diff 而非人工标注** (Terminus2 原文: "without task-specific hand labels")。动机：纯新增不会造成"状态坍缩"，只有覆盖型更新才会让旧状态连同"它何时有效"的上下文一起丢失。

判据"是谁说了算"——规则 / 额外模型 / 事件触发 (最易混，单列)：

Agent	触发机制	谁来判	关键点
A-Mem (F.4)	结构 diff 规则	规则 (图结构变化)	图 diff → overwrite_update / link_rewrite_update ；抑制 assistant 轮； 非相似度阈值，而是结构是否被改写
Terminus 2 (F.1)	指令 diff + 推断	规则 (diff) + 轻量推断	对相邻任务指令 diff，再从环境/执行证据推断变更类型，无人工标签
OpenHands (F.2)	代码覆盖检测	规则 (文件/逻辑覆盖)	后续 patch 推翻早先实现假设时记录
Memento-Skill / GAIA (F.3, G.4)	事件触发 + 额外 LLM	额外模型 (judge + updater) + 测试门	答错(事件)→ gpt-5.4-mini judge 判错 + 轨迹分析 → LLM updater 合成新 tip → post-update 单元测试 gate 放行

重点结论：只有 **GAIA (Memento-Skill)** 这条线是"额外模型判断 + 事件触发"——由 LLM judge 判对错、LLM updater 生成新 tip、单元测试门把关；其余三种都是**结构/diff 规则** (不是相似度阈值，也不是 prompt 顺手判)。原因是 GAIA 任务静态，"patch"不为追踪环境演化，而为**从失败中累积可复用经验**，所以天然由"失败事件"驱动而非 diff。

④ 四种 agent 上 base_memory / patch 的具体形态

Agent (域)	base_memory M_T 存什么	patch 存什么 / 触发条件	patch 检索
Terminus 2 (终端)	从历史终端轨迹蒸馏的 任务求解知识 (ledger: 目标、策略摘要、命令模式...)	相邻任务间的 策略转移 ：什么变了、哪个旧假设被推翻、观察到的适配、 "不要照抄"的旧具体值 (防 stale 复用)	比对当前任务契约变化 + 词法相似度
OpenHands (软件)	从历史轨迹蒸馏的 代码上下文 (文件、符号、约束、执行结果)	特征级记录 $\rho = (f, s, b^-, b^+, c, r, \Delta)$ ：	语义 + 结构 (文件路径/精确重叠)

Agent (域)	base_memory M_T 存什么	patch 存什么 / 触发条件	patch 检索
		受影响文件、符号、改前/改后行为、需保留的约束、原因、有界代码证据	混合打分
A-Mem (社交/偏好)	语义组织的 记忆图 (note + link)	note/relation 级 patch, 仅非增量 (overwrite_update / link_rewrite_update), 记 note 改前/改后状态; 单独用 all-MiniLM-L6-v2 嵌入存于独立 patch 检索器	嵌入检索 top- m (默认 2), 按时序排序
Memento-Skill (GAIA 工具)	全局 TIP.md 技能记忆	版本化 tip 家族 (存整份快照而非文本 diff, 用 lineage 指针表达 patch 语义), 记 tip 更新、触发失败、原因	BM25 取 top- k (默认 2)

⑤ 一个具体例子 (PersonaMem 偏好演化)

用户起初偏好"主流超市、清晰标签、少特殊食材", 后来转为"逛国际市场、尝试 sumac / tahini 等陌生食材"。

- **base_memory** 合并后只剩最新信念"爱逛国际市场"——但若问"为什么变 / 之前怎样", 证据已丢。
- **patch** 则记下 C^- ="偏好主流超市"、 C^+ ="探索国际市场"、 r ="想拓展尝试"、 e =触发的那几轮对话。
- 当题目是"周六下午去哪买菜"这类**时序轨迹题**时, 检索该 patch → agent 用**更新后**的偏好答对 (选国际市场), 而非被旧状态误导。

3. 关键实验结果

评测覆盖 **8 个主流 backbone**: GPT-5.5 / Gemini-3.1-Pro / Kimi-K2.6 / Deepseek-V4-Pro / GLM-5.1 / MiniMax-M2.7 / Qwen3.6-27B / Gemma4-31B。两套指标: **Step accuracy** (单个演化实例的平均准确率) 与 **Chain accuracy** (演化链每一步都要做对, 更严格)。

3.1 当前 agent 在 EvoArena 上很吃力

评测	当前 agent 平均准确率
EvoArena (terminal + software + social)	仅 39.6%

- 说明"动态演化环境"对现有 agent 是**真实的、未被满足的挑战**——尤其 SWE-Chain-Evo 的 chain accuracy 平均仅 ~10%

3.2 EvoMem 的增益 (按域 · Step / Chain 平均)

子基准	Step (基线→EvoMem)	Chain (基线→EvoMem)
Terminal-Bench-Evo	43.6 → 46.0 (+2.4)	21.5 → 27.6 (+6.1)
SWE-Chain-Evo	27.9 → 28.3 (+0.4)	10.0 → 12.1 (+2.1)

子基准	Step（基线→EvoMem）	Chain（基线→EvoMem）
PersonaMem-Evo	47.3 → 49.0 （+1.7）	40.0 → 43.2 （+3.2）
EvoArena 总平均	—	chain +3.7

外溢到标准基准（证明"记忆进化"是通用增益，而非只对自家基准奏效）：

标准基准	EvoMem 增益
GAIA	+6.1%（论文摘要；网站表 65.8→72.3, +6.5）
LoCoMo	+4.8%（论文摘要；网站表 39.7→43.0, +3.3）

摘要给的是平均增益口径，下面是 Table 4 的逐模型完整结果。

GAIA（agent = Memento-Skill，100 题平衡子集，LLM-judge 准确率）：

模型	Base	+EvoMem	Δ
GPT-5.5	83.0	83.0	+0.0
Gemini-3.1-Pro	57.0	65.0	+8.0
Gemma4-31B	45.0	54.0	+9.0
Deepseek-V4-Pro	70.0	80.0	+10.0
GLM-5.1	70.0	77.0	+7.0
Qwen3.6-27B	70.0	75.0	+5.0
平均	65.8	72.3	+6.5

LoCoMo（agent = A-Mem，exact match）：

模型	Base	+EvoMem	Δ
GPT-5.5	32.9	33.9	+1.0
Gemini-3.1-Pro	21.1	28.6	+7.5
Gemma4-31B	52.3	55.2	+2.9
Deepseek-V4-Pro	52.0	56.5	+4.5
Kimi-K2.6	54.0	57.7	+3.7
Qwen3.6-27B	26.0	26.3	+0.3
平均	39.7	43.0	+3.3

GAIA 实验详解（附录 G.4）

⚠️ 没有做 **GAIA2**：GAIA2（Froger et al. 2026，动态/异步环境基准）在论文中仅作为相关工作出现在 §1、Table 1、§2，用来论证“已有动态基准多是任务刷新/异步事件、很少测持续版本演化”。正式实验只用静态 **GAIA**。

任务设定

- **Agent**：Memento-Skill（支持复杂工具调用的 agent）。
- **数据**：用仓库划分 `gaia_data/data/split_by_level_60_40/train`，含 **100 个跨难度均衡的 GAIA 实例**。选它是因为比 held-out test 更大、更均衡。
- **关键改动**：原版 Memento-Skill 用该 train 集学技能、在 test 集评测；这里目的不同——在同一组固定任务上对比“同一个 **Memento-Skill agent 有无 EvoMem**”，从而单独隔离“EvoMem 是否提升了任务求解经验的复用”。
- **指标**：LLM-judge 准确率，judge 用 `gpt-5.4-mini` 判最终答案对错。

Base vs EvoMem 唯一差别 = 记忆检索层（求解 + 优化回路完全一致）

- 两者都用 **read-write-optimize** 协议：一轮反馈（`--optimize-attempts 1`）+ 开启 **post-update 单元测试 gate**。
- **Base**（`--learning-tips-mode baseline`）：只注入全局 TIP.md，**不检索任务专属 patch 记忆**。
- **EvoMem**（hybrid tip pipeline）：保留同一份全局 TIP.md，**额外用 BM25 从 MemGit 存储检索 top-2 任务版本化 tip**。

如何判断“是否增量 / 何时写新 patch”（GAIA 这条线特殊，附录 F.3）

- **不是 diff / 相似度规则，而是“事件触发 + 额外 LLM”**：只在出现可复用学习信号时才写新 tip 版本——典型是先答错 → judge 反馈 + 执行轨迹分析。
- 判断链：答错（事件）→ `gpt-5.4-mini judge` 判错 + 轨迹失败分析 → **LLM updater 合成新 tip** (τ_{T+1}, m_{T+1}) = $U(x, j, \tau_T, m_T)$ → **post-update 单元测试 gate 放行** → append 进版本家族。
- 为什么不用 diff：GAIA 任务静态、环境不演化，patch 不是为“追踪环境变化”，而是为从失败中累积可复用经验，所以天然由“失败事件”驱动而非记忆改写检测。
- “非增量”在这里几乎天然成立：每个新 tip 版本就是对上一版 TIP.md 的改写/重释（revise/reinterpret），patch 语义由 lineage 指针 p_t + 差异元数据 Δ_t 表达，存的是**整份快照**而非文本 diff。

结果解读（65.8 → 72.3，+6.5）

- **GPT-5.5 已 83% 触顶** → +0.0（没有提升空间）。
- **中等能力模型提升最大**：Deepseek-V4-Pro **+10**、Gemma4-31B **+9**、Gemini-3.1-Pro **+8**、GLM-5.1 **+7**、Qwen3.6-27B **+5**。
- 说明 patch 记忆的价值在于给“有能力执行但缺经验复用”的模型补上**历史教训**；天花板模型与极弱模型受益都更有限。
- 意义：GAIA 本身静态、不演化，EvoMem 仍有显著增益 → 证明“记忆=可检索的更新历史”是**通用机制**，而非只为演化环境定制。

关键亮点：

1. **Chain 增益 > Step 增益**——EvoMem 真正帮 agent 跟踪“演化链条”，而不仅是单步提分（Terminal chain +6.1 vs step +2.4）
2. 不只在 EvoArena 上有效，还能提升通用 **GAIA / LoCoMo**——“记忆进化”是可迁移的通用能力
3. 增益因 **backbone** 而异：弱模型（Gemma4-31B、Qwen3.6-27B）在 chain 上提升更明显，少数强模型在个别域出现轻微回退（如 GLM-5.1 在 PersonaMem chain 上 -3.7）

3.3 机理分析：EvoMem 何时 / 为何有效

- 被"操作化"时增益最大：Terminal-Bench-Evo 上，当 patch 真正被采纳（patch uptake≠0、改变了 agent 的计划/命令）时，增益从 +2.6% 升到 **+8.3%**
- 减少行为回退：SWE-Chain-Evo 上 **PASS_TO_PASS** 失败率从 **9.09%** 降到 **6.32%**——更好地保住了早期里程碑引入的行为
- 对"需时序/分散证据"的推理帮助最大：PersonaMem-Evo 上"时序轨迹 + 多模式综合"类问题 **+5.2%**——正是单一合并记忆最容易丢证据的设定
- 改善证据捕获：行级偏好证据捕获率 **72.5% → 74.9%**，在时序/多模式问题上提升最大
- 效率洞察：token 用量不是能力的可靠代理——Kimi-K2.6、Gemma4-31B 用低于均值的 token 即达强准确率，而 GPT-5.5 在 Terminal 上最强但 token 成本远高 → 主张"演化型基准应同时报告准确率与推理效率"

4. 对领域的影响 / 后续方向

🌟 学界 / 工业影响

1. 把"演化"视角引入评测与记忆——补上静态评测的盲区
 - 此前 agent 评测默认静态环境，EvoArena 首次系统建模"环境随时间演化"
 - 与 [03-agent-world/22-agentic-env-survey](#) 的"环境演化"主题呼应，但 EvoArena 聚焦评测 + 记忆
2. EvoMem —— "记忆即演化历史"的范式
 - patch-based 记忆把"环境怎么变"显式编码进记忆
 - 与 [05-skillrl](#) 的"存模式不存轨迹"形成对照——一个存演化历史、一个存行为模式
 - 直接呼应本库"记忆进化"主题（Memory Evolution）
3. NUS 主导、最新（2026-06）的前沿方向
 - 提示 2026 H2 的研究焦点正从"agent 能力进化"扩展到"环境/记忆的演化建模"
 - 作者阵容横跨 NUS / MIT / Salesforce(Recursive) 等，反映"动态评测"正成为跨机构共识

⚠️ 局限

- EvoMem 在 EvoArena 上的总体平均增益（摘要口径 **+1.5% step**）相对温和，且 **SWE-Chain-Evo** 几乎不涨（**step +0.4**）——说明软件演化仍是开放难题
- 个别强模型在某些域出现**负增益/回退**（如 GLM-5.1 在 PersonaMem chain -3.7）——patch 检索与采纳的稳定性待提升
- patch 序列会随演化变长，长程记忆管理与检索成本待解
- 三域（terminal/software/social）之外的演化类型覆盖有限

💡 揭示的趋势

1. 动态环境评测成为新前线——静态基准（GAIA/HLE）已不足以反映真实部署
2. 记忆 = 演化历史——把环境演化显式编码进结构化记忆，是 robust agent 的关键
3. Chain-level（演化链）评测——评的不是单步，而是"持续跟随演化"的能力

5. 资源

- **arXiv:** <https://arxiv.org/abs/2606.13681> (v1 2026-06-11, v2 2026-06-17, [cs.CL](#))
- **HF Papers:** <https://huggingface.co/papers/2606.13681>
- **GitHub:** <https://github.com/Aiden0526/EvoArena>
- **项目页:** <https://aiden0526.github.io/EvoArena/>
- **数据:** HF Collections Aiden0526/evoarena
- **机构:** NUS (主导) · SMU · UW · UCL · UPenn · NTU · Recursive · MIT (一作 Jundong Xu, 通讯 Zhiyuan Hu)
- **基准:** EvoArena = Terminal-Bench-Evo + SWE-Chain-Evo + PersonaMem-Evo (自建, 均在现有基准上"演化化"改造); 外加 GAIA · LoCoMo
- **被评 Agent:** Terminus 2 · OpenHands · A-Mem · Memento-Skill